A Mini Project Report Of

PCC-201-COM : Data Structures

Submitted by
Yash Goraksh Mapari (SEC B - 20)

Under the Guidance of
Prof. Tejal Harshal Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University
[2026-27]

BHARATI VIDYAPEETH
COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the "Mini Project Report" submitted by Yash Goraksh Mapari (20), on "Implementation of a Basic Version of a Web Browser's Back Button Functionality using Stack" is work done by him and submitted during the 2026–2027 academic year, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Engineering, at Bharati Vidyapeeth's College of Engineering, Lavale, Pune.

Prof. Tejal Harshal Patil
(Guide)

Dr. L. V. Patil
Head of Computer Engg. Department

ACKNOWLEDGEMENT

It gives us immense pleasure to express our gratitude and thanks to all those who have helped in the timely completion of the Mini-Project work.

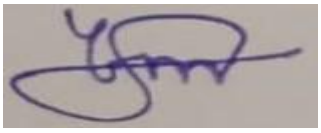
We have great pleasure in expressing our deep sense of gratitude and indebtedness to our Guide, **Prof. Tejal Harshal Patil**, for her invaluable guidance, constant supervision and sustained encouragement throughout the tenure of this Mini Project work.

We would like to express our thanks to **Dr. L. V. Patil**, Head of Department, Department of Computer Engineering, Bharati Vidyapeeth's College of Engineering, Lavale, Pune, for his constant support and encouragement.

We would like to express our thanks to **Prof. Dr. R. N. Patil**, Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune, for permitting us to take up the project work. We are also thankful to our parents and friends for their kind help and support throughout the course.

Yash Goraksh Mapari

Student Name



ABSTRACT

The Back button of a web browser is one of the most familiar real-world applications of the Stack data structure. A stack follows the Last-In, First-Out (LIFO) principle, which matches exactly how a user expects navigation to behave: pressing Back must take the user to the page that was visited most recently before the current one.

In this project, the Back and Forward functionality of a browser is simulated using two stacks — a *back_stack* that stores the pages visited earlier and a *forward_stack* that stores the pages undone by the Back button. When a new page is visited, the current page is pushed onto the back_stack and the forward_stack is cleared. When Back is pressed, the current page is moved to the forward_stack and the top of the back_stack becomes the current page.

Every operation uses only push and pop on the top of a stack and therefore runs in constant time, O(1). The project is implemented in Python using an object-oriented approach with a menu-driven interface, and it strengthens the understanding of stack operations, LIFO behaviour, state management and real-world problem modelling.

CONTENT

Sr. No.	Content	Page No.
	Acknowledgement	i
	Abstract	ii
	Content	iii
	List of Figures	iv
1	Introduction a) Overview b) Problem Statement c) Objective	1
2	Literature Survey	2
3	UML Diagram	3
4	System Requirement	4
5	System Design a) ER Diagram b) System Code	5
6	Result	9
7	Application	11
8	Future Scope	12
9	Conclusion	13
10	References	14

LIST OF FIGURES

Figure	Description	Page No.
Fig 3.1	Use Case Diagram	3
Fig 5.1	ER Diagram	5
Fig 5.2	Working of Back Operation (Stack States)	6
Fig 5.3	System Code	8
Fig 5.4	Result	10

INTRODUCTION

1.1 Overview

The project "Implementation of a Basic Version of a Web Browser's Back Button Functionality using Stack" demonstrates how the navigation history of a browser can be managed efficiently with the Stack data structure. A Stack follows the Last-In-First-Out (LIFO) principle, meaning the element inserted most recently is the first one to be removed. This is exactly the behaviour required by a Back button: the page visited last is the page that must be shown first when the user goes back.

The implementation uses two stacks — *back_stack* and *forward_stack* — together with a variable *current* that holds the page being displayed. Whenever the user visits a new page, the current page is pushed onto the back_stack. When the user presses Back, the current page is pushed onto the forward_stack and the page on top of the back_stack is popped and displayed. Pressing Forward performs the reverse movement. Only the basic stack operations (push and pop) are used.

The project is developed in Python using an object-oriented approach. It also includes a menu-driven interface that allows the user to visit pages, move back and forward, view the current page and display the entire history, making it easy to test and visualise the working of the algorithm.

1.2 Problem Statement

To implement a basic version of a web browser's Back (and Forward) button using Stack data structures and their basic push and pop operations, supporting visit, back, forward, current-page, display-history and back-button-status operations through a menu-driven Python program.

1.3 Objective

The main objective of this project is to design and implement the Back button functionality of a web browser in Python, demonstrating how a stack can model real-world navigation history. The project focuses on performing the essential operations — visit, back, forward and display — by utilising the basic stack operations push and pop.

Through this implementation, the project aims to enhance the understanding of the LIFO principle and of how a simple abstract data type can solve an everyday problem. By developing a menu-driven, object-oriented program, the project also improves practical coding skills, logical thinking and problem-solving ability.

LITERATURE SURVEY

Navigation history is a fundamental feature of every web browser. Whenever a user moves between pages, the browser must remember the order of visited pages so that the user can return to them. This requirement maps naturally onto the stack data structure, which is why the “Back button” is one of the most widely quoted examples when the Stack is introduced in data structures courses.

Two main approaches are commonly discussed for implementing browser history:

- **Two-Stack Method:** One stack stores the pages that can be reached using Back, and a second stack stores the pages that can be reached using Forward. Every operation affects only the tops of the stacks and takes O(1) time.
- **Array / Pointer Method:** All pages are stored in a single list together with an index to the current page. Back and Forward move the index; visiting a new page discards every entry after the index. This approach is used in the well-known problem *Design Browser History* (LeetCode 1472) and is closer to the way real browsers keep their session history.

This project adopts the two-stack method, because it demonstrates the LIFO property of a stack directly and requires only push and pop operations. A key rule observed by all real browsers is also implemented: **visiting a new page after going back erases the forward history**, since the user has started a new navigation path.

Implementing the Back button with stacks not only strengthens the understanding of the Stack abstract data type but also provides a hands-on framework for analysing time and space complexity, state management and object-oriented design in Python. The same idea appears in many other systems, such as the Undo feature of text editors and the activity back stack of mobile applications.

UML DIAGRAM

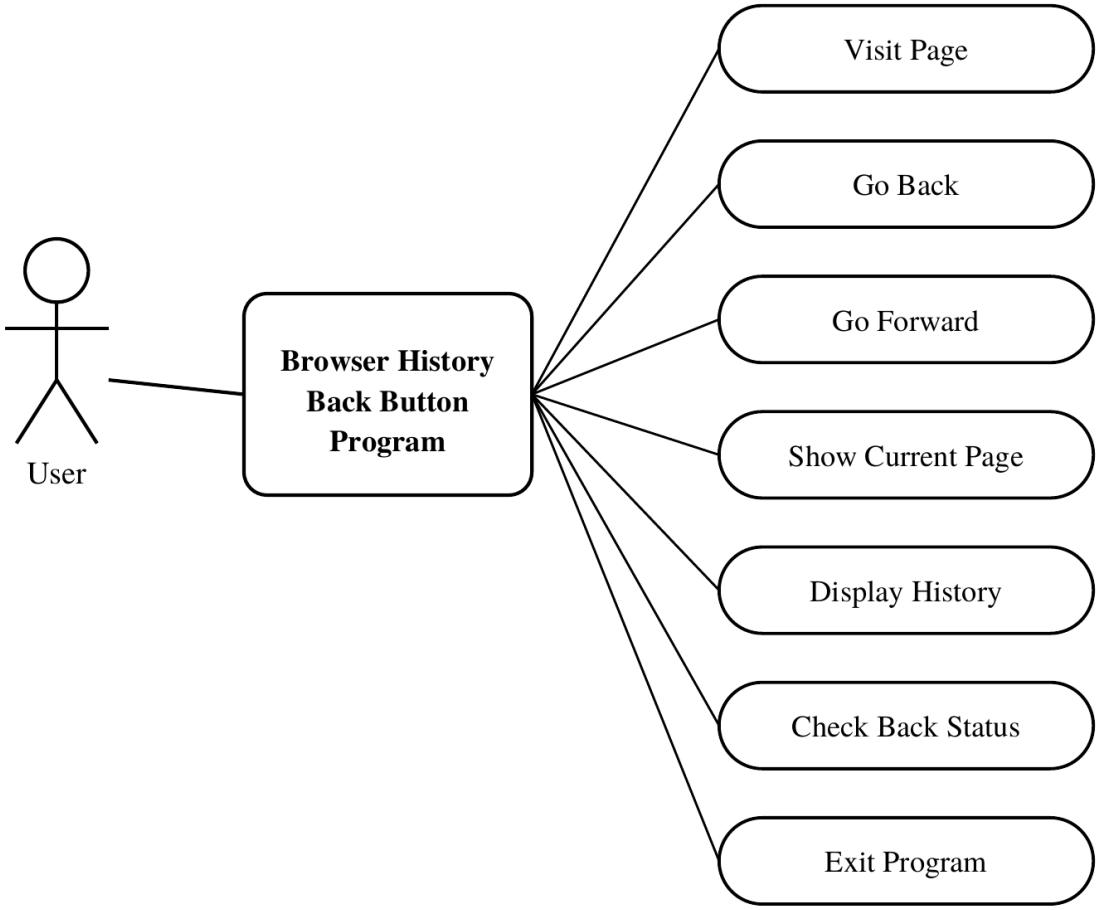


Fig 3.1: Use Case Diagram

SYSTEM REQUIREMENT

Hardware Requirements

Component	Minimum Requirement	Recommended
Operating System	Windows 7 / 8 / 10 / 11 (64-bit)	Windows 11 (64-bit)
Processor	Intel Core 2 Duo or equivalent	Intel Core i3 or higher
RAM	512 MB	2 GB or higher
Processor Speed	2.3 GHz or higher	3.0 GHz or higher
Hard Disk	1 GB free space	5 GB or more free space

Software Requirements

Software	Version / Notes
Python	3.7 or higher
IDE / Environment	Jupyter Notebook / PyCharm / VS Code
Required Libraries	None (uses built-in Python features only)

SYSTEM DESIGN

Entity Relationship Diagram

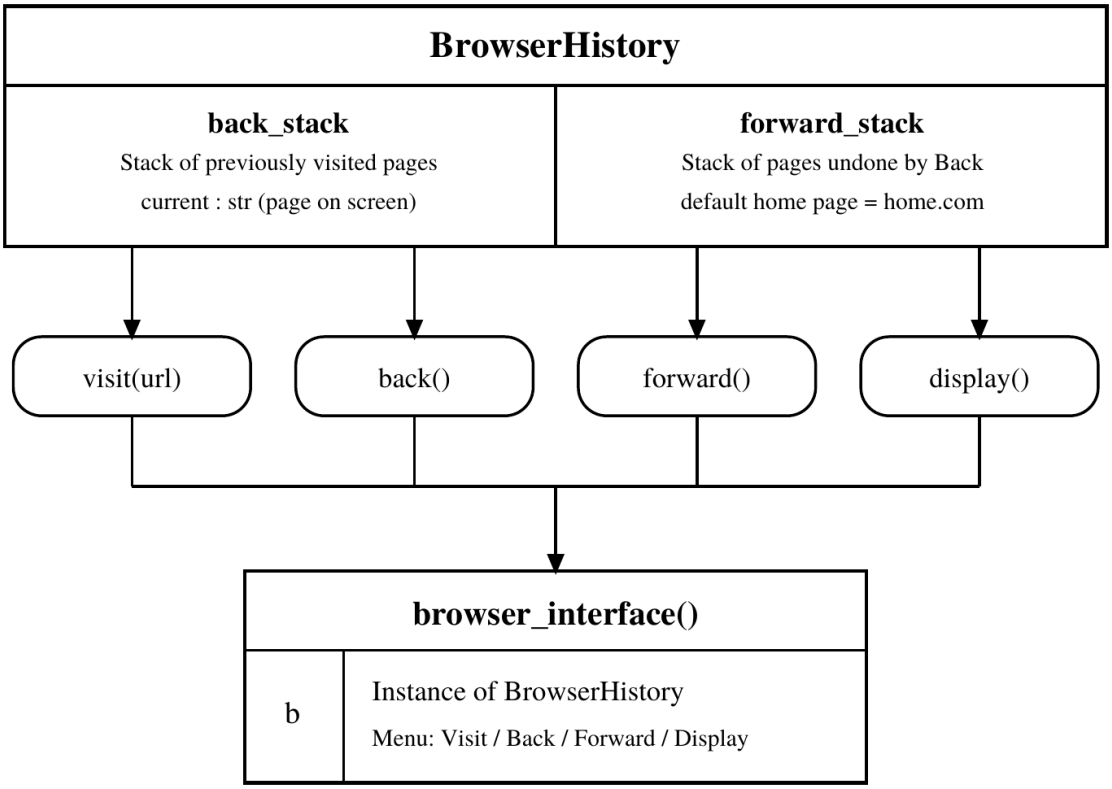


Fig 5.1: ER Diagram

Working Principle

The figure below shows how the two stacks and the current page change during a typical session. The home page is the first entry in the history.

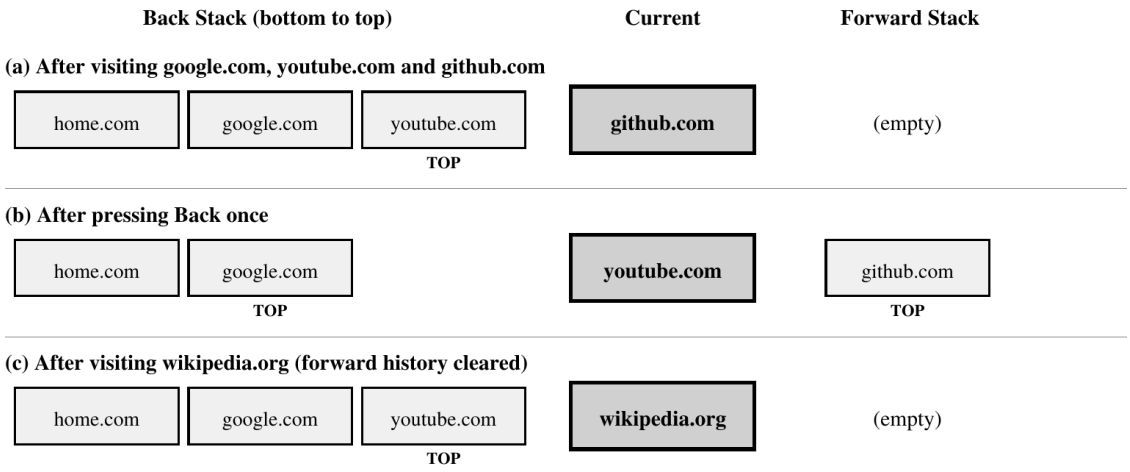


Fig 5.2: Working of Back Operation (Stack States)

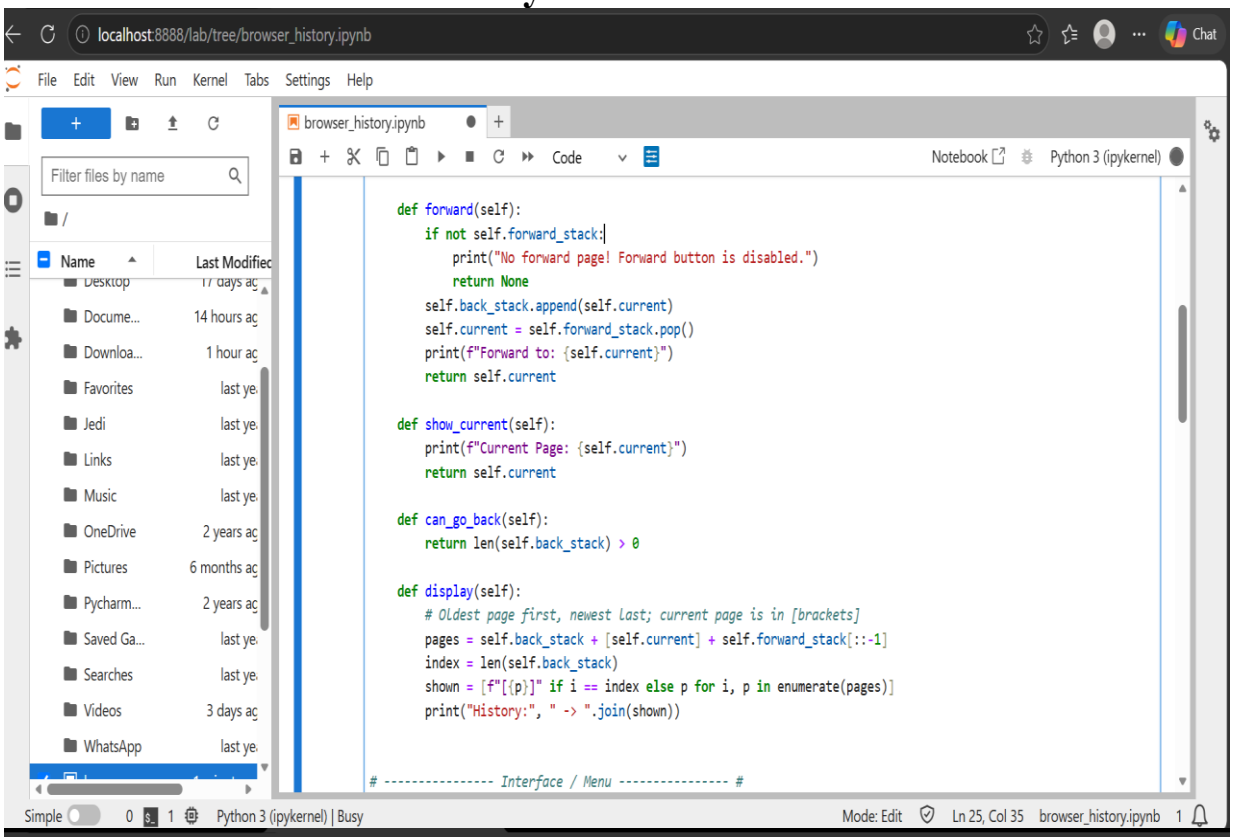
Algorithm

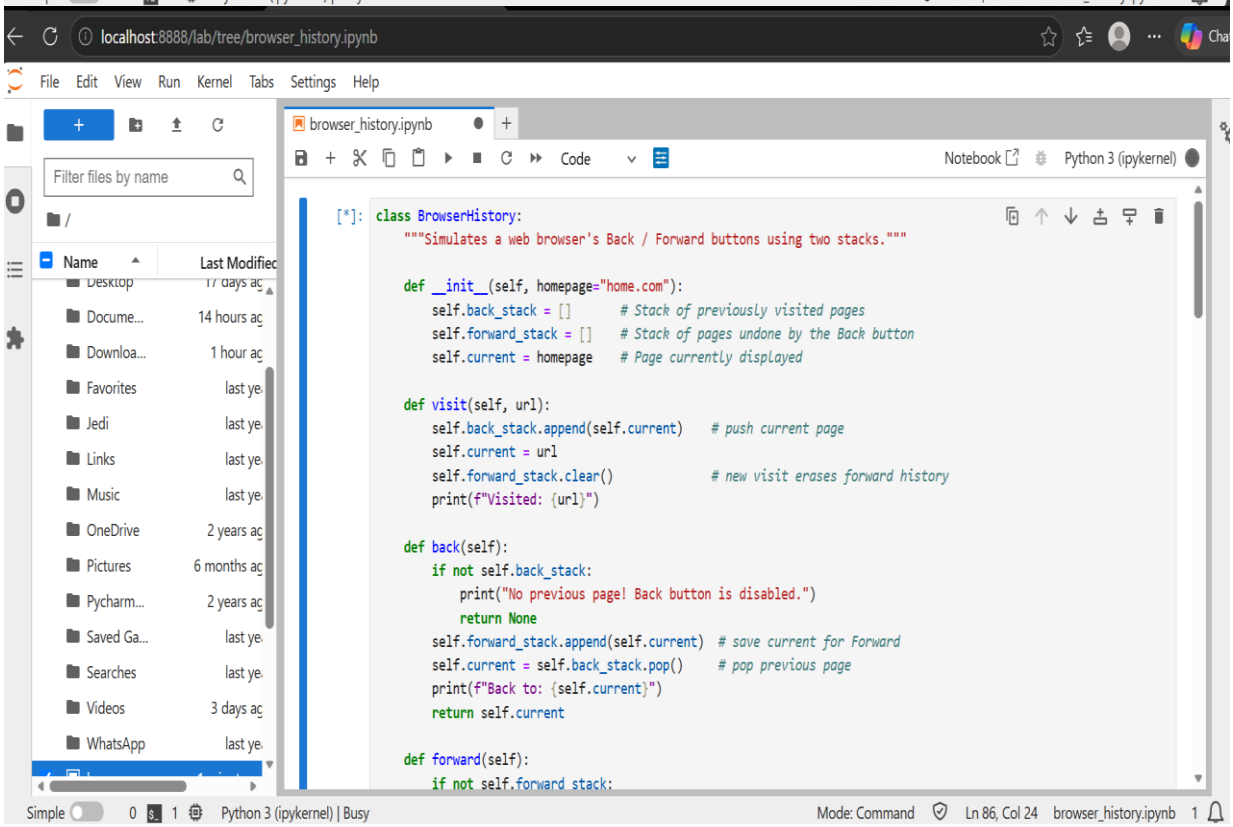
- **visit(url):** push *current* onto back_stack; set *current* = url; clear forward_stack.
- **back():** if back_stack is empty, display a message and stop; otherwise push *current* onto forward_stack and set *current* = pop(back_stack).
- **forward():** if forward_stack is empty, display a message and stop; otherwise push *current* onto back_stack and set *current* = pop(forward_stack).
- **display():** print back_stack from bottom to top, then the current page, then forward_stack from top to bottom.

Complexity Analysis

Operation	Time Complexity	Space Complexity
visit(url)	O(1) (clear is O(k) only for discarded forward pages)	O(1)
back()	O(1)	O(1)
forward()	O(1)	O(1)
show_current()	O(1)	O(1)
display()	O(n)	O(n)

System Code


localhost:8888/lab/tree/browser_history.ipynb
File Edit View Run Kernel Tabs Settings Help
browser_history.ipynb
Code
Notebook
Python 3 (ipykernel)
Filter files by name
Name
Last Modified
def forward(self):
if not self.forward_stack:
print("No forward page! Forward button is disabled.")
return None
self.back_stack.append(self.current)
self.current = self.forward_stack.pop()
print(f"Forward to: {self.current}")
return self.current
def show_current(self):
print(f"Current Page: {self.current}")
return self.current
def can_go_back(self):
return len(self.back_stack) > 0
def display(self):
Oldest page first, newest last; current page is in [brackets]
pages = self.back_stack + [self.current] + self.forward_stack[::-1]
index = len(self.back_stack)
shown = [f"[{p}]" if i == index else p for i, p in enumerate(pages)]
print("History:", " -> ".join(shown))
---------- Interface / Menu ----------
Simple
Python 3 (ipykernel) | Busy
Mode: Edit
Ln 25, Col 35
browser_history.ipynb


localhost:8888/lab/tree/browser_history.ipynb
File Edit View Run Kernel Tabs Settings Help
browser_history.ipynb
Code
Notebook
Python 3 (ipykernel)
Filter files by name
Name
Last Modified
[*]:
class BrowserHistory:
"""Simulates a web browser's Back / Forward buttons using two stacks."""
def __init__(self, homepage="home.com"):
self.back_stack = []
Stack of previously visited pages
self.forward_stack = []
Stack of pages undone by the Back button
self.current = homepage
Page currently displayed
def visit(self, url):
self.back_stack.append(self.current)
push current page
self.current = url
self.forward_stack.clear()
new visit erases forward history
print(f"Visited: {url}")
def back(self):
if not self.back_stack:
print("No previous page! Back button is disabled.")
return None
self.forward_stack.append(self.current)
save current for Forward
self.current = self.back_stack.pop()
pop previous page
print(f"Back to: {self.current}")
return self.current
def forward(self):
if not self.forward_stack:
Simple
Python 3 (ipykernel) | Busy
Mode: Command
Ln 86, Col 24
browser_history.ipynb

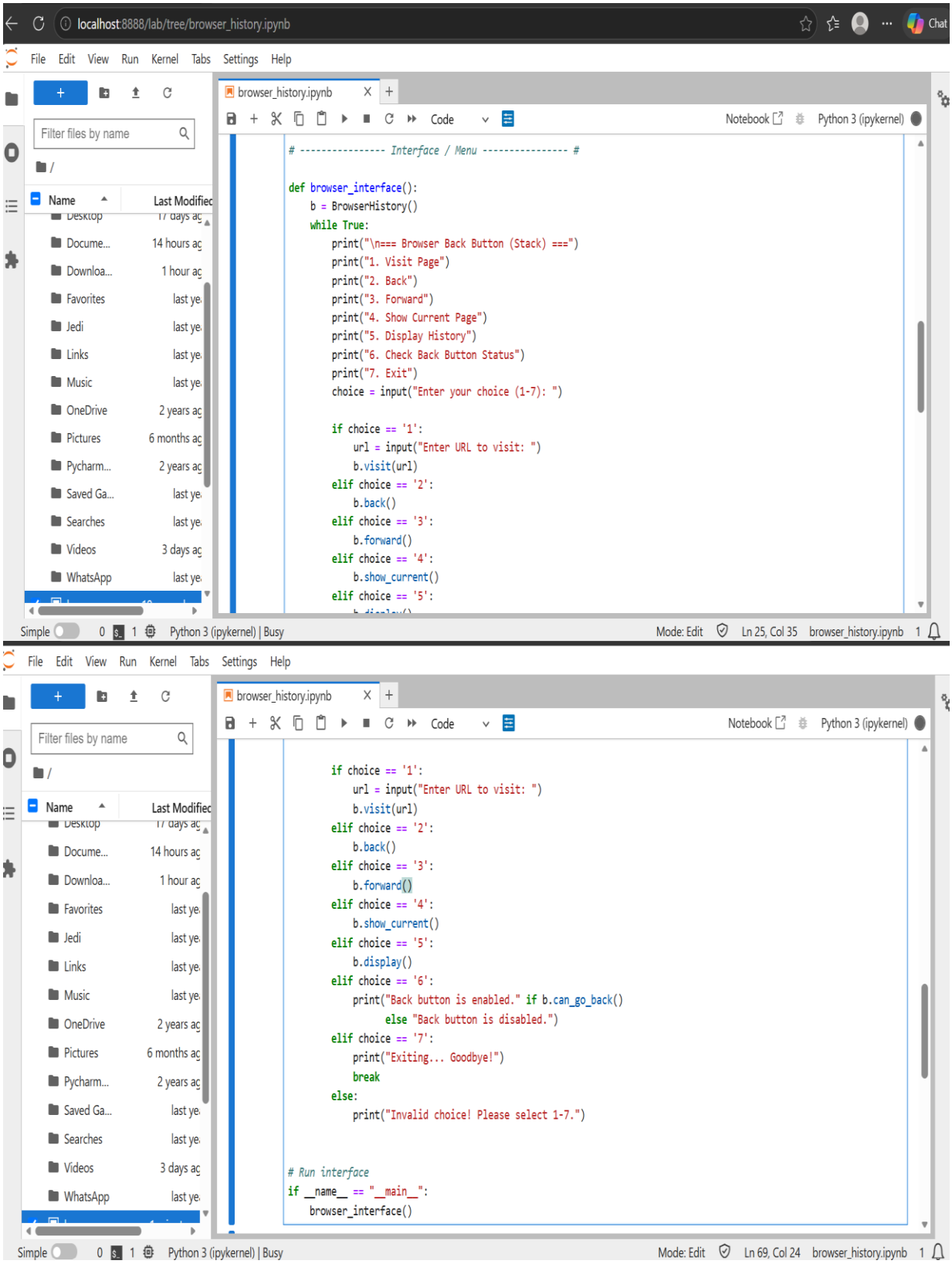


Fig 5.3: System Code

RESULT

The program was executed with a sequence of inputs that covers normal navigation, the Back and Forward buttons, clearing of forward history, disabled-button messages and an invalid menu choice. The output is shown below.

```
=== Browser Back Button (Stack) ===
1. Visit Page
2. Back
3. Forward
4. Show Current Page
5. Display History
6. Check Back Button Status
7. Exit
Enter your choice (1-7): 1
Enter URL to visit: google.com
Visited: google.com

=== Browser Back Button (Stack) ===
Enter your choice (1-7): 1
Enter URL to visit: youtube.com
Visited: youtube.com

=== Browser Back Button (Stack) ===
Enter your choice (1-7): 1
Enter URL to visit: github.com
Visited: github.com

=== Browser Back Button (Stack) ===
Enter your choice (1-7): 5
History: home.com -> google.com -> youtube.com -> [github.com]

=== Browser Back Button (Stack) ===
Enter your choice (1-7): 2
Back to: youtube.com

=== Browser Back Button (Stack) ===
Enter your choice (1-7): 2
Back to: google.com

=== Browser Back Button (Stack) ===
Enter your choice (1-7): 4
Current Page: google.com

=== Browser Back Button (Stack) ===
Enter your choice (1-7): 3
Forward to: youtube.com

=== Browser Back Button (Stack) ===
Enter your choice (1-7): 1
Enter URL to visit: wikipedia.org
Visited: wikipedia.org

=== Browser Back Button (Stack) ===
Enter your choice (1-7): 3
No forward page! Forward button is disabled.

=== Browser Back Button (Stack) ===
Enter your choice (1-7): 5
History: home.com -> google.com -> youtube.com -> [wikipedia.org]

=== Browser Back Button (Stack) ===
```

```
Enter your choice (1-7): 2
Back to: youtube.com

=== Browser Back Button (Stack) ===
Enter your choice (1-7): 2
Back to: google.com

=== Browser Back Button (Stack) ===
Enter your choice (1-7): 2
Back to: home.com

=== Browser Back Button (Stack) ===
Enter your choice (1-7): 2
No previous page! Back button is disabled.

=== Browser Back Button (Stack) ===
Enter your choice (1-7): 6
Back button is disabled.

=== Browser Back Button (Stack) ===
Enter your choice (1-7): 9
Invalid choice! Please select 1-7.

=== Browser Back Button (Stack) ===
Enter your choice (1-7): 7
Exiting... Goodbye!
```

Fig 5.4: Result

APPLICATIONS

The stack-based back functionality is not limited to web browsers. The same LIFO idea is used in many systems:

1. Computer Science Education and Interviews

- **Concept Demonstration:** A classic, easy-to-visualise example used to teach the LIFO principle and the practical use of stacks.
- **Skill Assessment:** Related problems such as "Design Browser History" are frequently asked in technical interviews to test data-structure selection and time-complexity reasoning.

2. Real-World Systems

Application	Description
Web Browsers	Back and Forward buttons let the user revisit previously opened pages in reverse order of visit.
Undo / Redo in Editors	Each edit is pushed onto an undo stack; Undo pops it and pushes it onto a redo stack, exactly like Back and Forward.
Mobile App Navigation	Android's activity back stack returns the user to the previous screen when the Back gesture or button is used.
File Explorers	The Back arrow in a file manager returns to the folders opened earlier, in reverse order.
IDE Code Navigation	"Navigate Back" jumps to the previous cursor location after "Go to Definition".
Expression Evaluation	Stacks are used for matching brackets and for evaluating postfix expressions.

FUTURE WORK

1. Robustness and Performance

- **URL Validation:** Check that the entered text is a well-formed URL before it is added to the history.
- **History Size Limit:** Restrict the back_stack to a fixed size (for example, the last 50 pages) and discard the oldest entries, as real browsers do.
- **Unit Tests:** Implement tests using Python's *unittest* or *pytest* for edge cases such as:
  - Pressing Back on the home page.
  - Pressing Forward when the forward stack is empty.
  - Visiting a new page after Back and checking that forward history is cleared.

2. Feature and Interface Expansion

- **Multiple Tabs:** Maintain a separate pair of stacks for every open tab.
- **Persistent History:** Save the history to a file or database so that it survives program restarts.
- **Timestamps and Page Titles:** Store a record (URL, title, time) for each page instead of just a string.
- **Type Hinting and Magic Methods:** Add type hints (e.g. *def visit(self, url: str) -> None*) and implement *__len__* and *__str__* for cleaner code.
- **Graphical Interface:** Build a GUI with Back and Forward buttons using Tkinter, and move the class and the menu function into separate modules.

CONCLUSION

The *BrowserHistory* class provides a successful and simple implementation of a web browser's Back and Forward functionality using two Last-In, First-Out (LIFO) stacks, *back_stack* and *forward_stack*. The program demonstrates clearly how the Stack abstract data type models a real-world navigation problem.

This implementation is particularly valuable because it:

- Demonstrates the practical use of the LIFO principle in an everyday application.
- Performs visit, back and forward operations in O(1) time using only push and pop.
- Correctly handles boundary cases such as an empty back stack, an empty forward stack and the clearing of forward history on a new visit.

The current code is functional and includes a basic menu-driven interface. Future improvements should focus on robustness (URL validation, history limits), feature completeness (tabs, persistent history, graphical interface) and formal testing, so that it can grow from a teaching model into a reliable navigation component.

REFERENCES

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

[2] LeetCode Problem 1472: Design Browser History.

[3] Python Software Foundation. *The Python Tutorial – Data Structures: Using Lists as Stacks.* docs.python.org.

[4] Mozilla Developer Network (MDN). *History API – Web APIs.* developer.mozilla.org.